N2: Performance Evaluation

Copyright © 1995-2021 Thomas M Conte

1

Other metrics of performance

- Power

Power(dynamic) = ½ * Capacitive load * Voltage^2 * Frequency

- Energy

Energy(dynamic) = Capacitive load * Voltage^2

- To reduce dynamic power:
  - Lower voltage
  - Lower frequency
- To reduce energy:
  - Lower voltage (frequency doesn't matter!)

2

Two Notions of (Time) Performance

Plane	DC to Paris	Top Speed	Passen-gers	Throughput (pmph)
Boeing 747	6.5 hours	610 mph	470	286,700
Concorde	3 hours	1350 mph	132	178,200

- Which has higher performance?
- Time to deliver 1 passenger?
- Time to deliver 400 passengers?
- In a computer, time for 1 job called Turnaround Time or Execution Time
- In a computer, jobs per day called Throughput or Bandwidth

How do you measure performance?

- Build it!
  - Costly
- Model
  - Simulation
  - Mathematical (Queuing, Markov, Petri)

"The purpose of computing is insight, not numbers"

- Richard Hamming (1915-98)

from Numerical Methods for Scientists and Engineers, 1962

Types of simulation

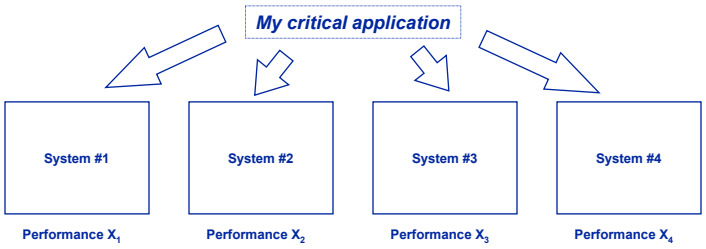
- Circuit level
  - Example: SPICE
- Gate level
  - Many examples, usually has 4 states (0, 1, X, Z)
- Both of the above require a detailed, completed design- very costly
- Functional level: sometimes called a "Timer"
  - For a given set of instructions, determine how many cycles they take to execute
  - Usually inaccurate in absolute terms, but accurate in relative terms:

    Ratio of (architecture#1 runtime) / (architecture #2 runtime) is accurate, but absolute runtimes of #1 and #2 are not

5

What workloads do we use to evaluate performance?

- Run your critical application on X computers



Then…Pick max of X_i …

6

Benchmarks

- Of course, that's expensive and difficult
- Solution: benchmarks
- Benchmark suites:
  - SPEC89, SPEC92, SPEC95, SPEC2K, SPEC2k6, SPEC2017 (for workstations, PCs) Includes code and inputs
  - EEMBC (for embedded systems, phones)
  - GEEKBENCH (for phones and PCs)
  - Transaction Process Performance Council TPC-A, B, C, D, H (web/database servers) - No code, just a "specification"
  - High-Performance LINPACK (HPL), High-Performance Conjugate Gradient (HPCG)

PERF-7

7

Benchmarking: three purposes

- The purpose of benchmarking depends on who you talk to:
  - Marketing: Sell computers
  - The Architect: Evaluate performance of potential designs
  - The Users: Be our proxy, run our applications on new systems so we don't waste our money or our time
- …For this class, we will safely ignore the marketing purpose!

PERF-8

8

CPU run time equation:

$$\text{CPU time} = \text{clock cycle count} \times \text{Cycle Time}$$

$$= \frac{\text{clock cycle count}}{\text{clock rate (Mhz)}}$$

$$\text{Cycles per instruction} = \text{CPI} = \frac{\text{clock cycle count}}{\text{Instruction Count}}$$

$$\Rightarrow \text{clock cycle count} = \text{CPI} \times \text{IC}$$

$$\therefore \boxed{\text{CPU time} = \text{IC} \times \text{CPI} \times \text{CT}}$$

"Newton's law" of microarchitecture

h/w tech and organization
organization and ISA
Program, ISA and compiler

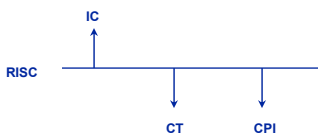
- To improve CPU time:
  - Decrease Instruction Count (IC)
    - Good compiler
    - Better software algorithms
  - Decrease CPI (increase "IPC", a.k.a. *inst. level parallelism, or ILP*)
    - Fancy hardware (e.g., caches, branch prediction, pipelining, superscalar)
    - Good compiler
  - Decrease CT
    - Deeper pipelining & really good circuit design
    - Technology scaling (ie, faster transistors)
    - Simple ISA; Less aggressive ILP (simple microarchitecture)

PERF-11

11

The real story on RISC vs. CISC

- RISC: Simple instructions
  - Takes a lot of them to do anything: Increases IC
  - Easier to build hardware: Decreases CT
  - Easier to parallelize: Decreases CPI
  - Net effect:
    - Need a lot of memory to hold program, but…
    - Runs faster if Inc(IC) is less than Dec(CT) and Dec(CPI)



CISC:

ADD (C), (A), (B)

RISC:

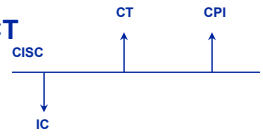
LD r1, (A)
LD r2, (B)
ADD r3, r1, r2
ST r3, (C)

PERF-12

12

Now CISC...

- CISC: Big, honking complex instructions
  - Takes very few to do anything: Decreases IC
  - Easier to program by hand
  - Harder to build fast hardware: Incr. CT
  - Harder to parallelize: Increases CPI
  - Net effect:
    - Memory efficient
    - Runs faster if Dec(IC) is greater than Inc(CT) and Inc(CPI)



- Retrospective:
  - If memory is expensive, people hand code machines, and compilers are terrible => use CISC
    - See the 60's and 70's!
    - …or Java bytecode in smartphones today!
  - If memory is inexpensive, no one hand codes, and compilers are terrific => Use RISC

PERF-13

13

Definitions: Speedup

- Time performance is in units of things per sec
  - bigger is better
- If we are primarily concerned with response time, smaller is better:

$$\text{performance(x)} = \frac{1}{\text{execution time(x)}}$$

" X is *n* times faster than Y " means (*n* is the "speedup"):

$$n = \frac{\text{Performance of X}}{\text{Performance of Y}} = \frac{\text{Execution time of Y}}{\text{Execution time of X}}$$

PERF-14

14

Why is speedup "better" than runtime?

- CPU time = IC * CPI * CT
  - But for parallel architectures that use synchronization, IC and CPI can vary depending on the architecture even for the same benchmark
  - Why: waiting on an event is not doing "useful work" and shouldn't "count"
- All that matters is going faster than before!

PERF-15

15

Amdahl's law– originally (1967!)

- For over a decade [ha!] prophets have voiced the contention that the organization of a single computer has reached its limits and that truly significant advances can be made only by interconnection of a multiplicity of computers in such a manner as to permit co-operative solution...The nature of this overhead (in parallelism) appears to be sequential so that it is unlikely to be amenable to parallel processing techniques. ...At any point in time it is difficult to foresee how the previous bottlenecks in a sequential computer will be effectively overcome.

PERF-16

16

Amdahl's law

- **Performance Improvement ("*speedup*") is limited by the part you cannot improve**

$$\text{Speedup} = \frac{\text{Performance of task with gizmo}}{\text{Performance of task without gizmo}} \quad \textit{or} \quad \text{Speedup} = \frac{\text{Execution time of task without gizmo}}{\text{Execution time of task with gizmo}}$$



S_e = best case speedup from enhancement ("gizmo") when used all the time
f_e = fraction of TIME that gizmo can enhance (i.e., fraction of time gizmo used)

$$(s_{overall} =)\ \text{Speedup}_{overall} = \frac{\text{Execution time of task without enhancement ("gizmo")}}{\text{Execution time of task with enhancement ("gizmo")}}$$

$$= \frac{\text{Time}_{old}}{\text{Time}_{old} \times (1 - f_e) + \text{Time}_{old} \times \frac{f_e}{S_e}}$$

$$= \frac{1}{(1 - f_e) + \frac{f_e}{S_e}}$$

PERF-17

17

Amdahl's law example

- **You do simulation of jet plane wings**
  - **1 run takes 1 week on your fastest computer**
- **You get this ad in your mailbox:**
  - **The *Acme Hyperbole* is the largest supercomputer ever built, it has 100,000 processors (great!)**
  - **It costs $1 bazillion (not so great)**
- **Now, 1 week is 600,000 sec, so**
  - **You could run a simulation in 6 seconds, right?**

$$f = 1, s_{overall} = s = 100{,}000$$

  - **Well, not all of a program can be done at the same time**
    - **Data dependencies:**
      - **x = (…), followed by (…) = x * y**
    - **Control dependencies:**
      - **If *xxx* then *yyy* else *zzz***
    - **Say 80% of your program *is* parallelizable (pretty good)**

PERF-18

18

Amdahl's law example (cont.)

$S_e = 100,000$ since there are 100,000 processors to perform the parallel part
$f_e = 0.8$ (i.e., 80%) since 80% of the workload can be parallelized

$$\text{Speedup}_{\text{overall}} = \frac{1}{(1-f_e)+\frac{f_e}{S_e}}$$
$$= \frac{1}{(1-0.8)+\frac{0.8}{100000}}$$
$$\approx \frac{1}{0.2} = 5$$

- So approximately 5 times faster, or 33 hours
  - Not quite as great as one would hope
  - Worth a bazillion dollars? *(TRY 100 PROCS => 4.8 !!!)*

PERF-19

19

But wait…Gustafson's Law

- Since 20% of the time is spent in the sequential
  - …the rest of the processors could be used to do *another* jet plane wing simulation or a *more detailed* simulation
- Say it takes T1+T2 time to run a parallel program, where T1 is the time spent sequentially, and T2 is the time spent in parallel
- P processors, the *equivalent sequential time* would be…

  sequential time = T1+T2*P
- The ratio of these is the speedup…

  Speedup(P) = (T1+T2*P)/(T1+T2)
- Define α = *sequential fraction of a parallel program*

  = T1/(T1+T2)
- Then for an unlimited-sized problem,

  Speedup(P) = α + (1-α)*P
- Or…

PERF-20

20

Gustafson's law (cont.)

$$S(P) = P - \alpha*(P-1)$$

This is roughly linear with increasing P!

But only true if you never "run out of" parallel work to do

Note also that "f" from Amdahl's law is ≠ (1-α) from Gustafson's law…

- Amdahl "f": fraction of a sequential fixed program that is parallelizable
- Gustafson "α": fraction of a parallel scalable program that is sequential

PERF-21

21

Amdahl vs. Gustafson

- Amdahl: the speedup for a fixed amount of work is limited by the part you cannot speed up
- Gustafson: If you have other work to do, however, you can accelerate that, too… until you run out of work to do

- For a benchmark, for simulation, for seeing if a new idea is better than an old, Amdahl matters
- But your new idea might give programmers new ideas!!!

PERF-22

22

Dealing with too many numbers

	Computer A	Computer B	Computer C
Program P1 (sec)	1	10	20
Program P2 (sec)	1000	100	20
Total time (sec)	1001	110	40

- A is 10 x faster than B for P1
- B is 10 x faster than A for P2
- A is 20 x faster than C for P1
- C is 50 x faster than A for P2
- etc.
- Total execution time gives clearest picture:
  - B is 1001/110 = 9.1 x faster than A for both programs
  - C is 25 x faster than A for both programs
  - C is 2.75 x faster than B for both programs
  - Which would you buy? (Answer: C is fastest, overall)
- *Arithmetic mean* of times is good too (A:500.5, B:55, C:20)

PERF-24

24

Example of means

	A	B
Prog 1	4	2
Prog 2	4	7
Hmean	4	3.11
Amean	4	4.5

Table holds rates in instructions per second

- Which is faster, A or B?
  - Consider running an average instruction from prog 1 followed by one from prog 2:
  - for A: (1/4 + 1/4) = 1/2
  - for B: (1/2 + 1/7) = 9/14

  Convert rates to times – then we can add the times together
  - A runs the two instructions faster (1/2 < 9/14), thus A is better
  - Now look at the harmonic mean (Hmean) vs. the arithmetic mean (Amean).
    - Hmean says A has a higher rate than B (4 vs. 3.11) so A is better
    - Amean says B has a higher rate than A (4.5 vs. 4) so B is better, but that's wrong!
- If you used the wrong method to combine the numbers, you would buy the slower machine!
- Note also that the definition of harmonic mean is just the average of the rates converted to times, then converted back to rates

PERF-25

25

More on combining numbers

- **Rules:**
  - **Use arithmetic mean to combine run times**

$$\bar{x} = \sum_{i=1}^{N} \text{weight}_i \times \text{time}_i = \frac{1}{N} \sum_{i=1}^{N} \text{time}_i$$

  - **Use harmonic mean to combine rates (e.g., IPC), because it actually combines them as times then converts back to a rate**

$$\overline{H} = \left(\sum_{i=1}^{N} \text{weight}_i / \text{rate}_i \right)^{-1} = \frac{N}{\sum_{i=1}^{N} \frac{1}{\text{rate}_i}}$$